

Python Interview Questions & Answers

Prepared for Interview Prep

Python Interview Questions & Answers

A complete reference covering core fundamentals, intermediate concepts, OOP, coding problems, and practical/applied topics commonly asked in Python interviews.

Section 1: Core Language Fundamentals

1. Difference between list, tuple, set, and dictionary

- **List** — ordered, mutable, allows duplicates. `my_list = [1, 2, 3]`
- **Tuple** — ordered, immutable, allows duplicates. `my_tuple = (1, 2, 3)`
- **Set** — unordered, mutable, no duplicates. `my_set = {1, 2, 3}`
- **Dictionary** — unordered (insertion-ordered since Python 3.7), mutable, key-value pairs, keys must be unique and hashable. `my_dict = {"a": 1, "b": 2}`

Tuples are faster than lists and can be used as dictionary keys (since they're hashable, as long as their elements are hashable). Lists cannot be dictionary keys.

2. Mutable vs immutable types — and the default argument gotcha

Immutable: int, float, str, tuple, frozenset, bool. Mutable: list, dict, set, custom objects (by default).

Classic gotcha — never use a mutable default argument:

```
def append_to(item, target=[]): # BAD: default list is shared
    across calls
    target.append(item)
    return target

print(append_to(1)) # [1]
print(append_to(2)) # [1, 2] <-- unexpected!
```

Fix:

```
def append_to(item, target=None):
    if target is None:
        target = []
    target.append(item)
    return target
```

The default value is evaluated **once** at function definition time, not on every call, so mutable defaults persist across calls.

3. is vs ==

- == compares **values** (calls `__eq__`).
- is compares **identity** (whether two references point to the same object in memory).

```
a = [1, 2, 3]
b = [1, 2, 3]
a == b    # True (same value)
a is b    # False (different objects)
a is a    # True
```

Small integers (-5 to 256) and interned strings may return True for is due to CPython's caching — but this is an implementation detail, not something to rely on.

4. Deep copy vs shallow copy

```
import copy

original = [[1, 2], [3, 4]]

shallow = copy.copy(original)    # copies outer list; inner lists
deep = copy.deepcopy(original)   # recursively copies everything

shallow[0].append(99)
print(original[0]) # [1, 2, 99] <-- affected because inner list is
print(deep[0])     # [1, 2]      <-- unaffected
```

A shallow copy creates a new outer object but keeps references to nested objects. A deep copy recursively duplicates every nested object.

5. *args and **kwargs

- *args collects extra **positional** arguments into a tuple.
- **kwargs collects extra **keyword** arguments into a dict.

```
def demo(*args, **kwargs):
    print(args)      # tuple, e.g. (1, 2, 3)
    print(kwargs)    # dict, e.g. {'x': 10, 'y': 20}

demo(1, 2, 3, x=10, y=20)
```

They're also used to unpack when *calling* a function: `demo(*my_list, **my_dict)`.

6. @staticmethod vs @classmethod vs instance methods

```
class Example:
    def instance_method(self):
        return f"called on instance {self}"

    @classmethod
    def class_method(cls):
        return f"called on class {cls}"

    @staticmethod
    def static_method():
        return "no access to self or cls"
```

- **Instance method** — takes `self`, operates on a specific instance.
- **Class method** — takes `cls`, operates on the class itself; often used

for alternate constructors.

- **Static method** — takes neither; it's just a regular function namespaced inside the class, used for utility logic related to the class.

7. List comprehensions vs generator expressions

```
squares_list = [x**2 for x in range(10)]    # builds the whole list
in memory
squares_gen  = (x**2 for x in range(10))    # lazy, yields one
value at a time
```

List comprehensions compute and store everything immediately. Generator expressions are lazy and memory-efficient, especially useful for large or infinite sequences, since values are produced on demand.

8. Generators and yield vs return

A generator function uses `yield` to produce a sequence of values lazily, pausing its state between calls.

```
def count_up_to(n):
    i = 1
    while i <= n:
        yield i
        i += 1

for num in count_up_to(5):
    print(num)
```

`return` exits a function and sends back a single value, ending execution. `yield` pauses the function, remembers its state, and resumes from that point on the next call — enabling iteration without holding the whole sequence in memory.

9. `__str__` vs `__repr__`

- `__str__` — human-readable string, used by `print()` and `str()`.
- `__repr__` — unambiguous, developer-facing representation, used by `repr()` and in the interactive shell; ideally it should look like valid Python code that could recreate the object.

```
class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __repr__(self):
        return f"Point({self.x}, {self.y})"

    def __str__(self):
        return f"({self.x}, {self.y})"
```

If `__str__` is not defined, Python falls back to `__repr__`.

10. Exception handling

```
try:
    result = 10 / 0
except ZeroDivisionError as e:
    print(f"Error: {e}")
except (TypeError, ValueError) as e:
    print(f"Type/Value error: {e}")
else:
    print("Runs only if no exception occurred")
```

```
finally:
    print("Always runs, exception or not")
```

Custom exceptions:

```
class InsufficientFundsError(Exception):
    pass

def withdraw(balance, amount):
    if amount > balance:
        raise InsufficientFundsError("Not enough balance")
```

11. Iterators vs iterables

- **Iterable** — an object you can loop over; implements `__iter__()`, which returns an iterator (e.g. lists, tuples, dicts, strings).
- **Iterator** — an object that implements both `__iter__()` and `__next__()`, and keeps track of its own state/position.

```
class Counter:
    def __init__(self, limit):
        self.limit = limit
        self.n = 0

    def __iter__(self):
        return self

    def __next__(self):
        if self.n >= self.limit:
            raise StopIteration
        self.n += 1
        return self.n

for num in Counter(3):
    print(num)    # 1, 2, 3
```

Section 2: Intermediate / Trickier Concepts

12. The GIL (Global Interpreter Lock)

The GIL is a mutex in CPython that allows only one thread to execute Python bytecode at a time, even on multi-core machines. It simplifies memory management (reference counting isn't thread-safe otherwise) but means CPU-bound multithreaded code doesn't get true parallelism. I/O-bound tasks (network calls, file I/O) still benefit from threading because the GIL is released during I/O waits.

13. Multithreading vs multiprocessing vs asyncio

- **Threading** (threading module) — good for I/O-bound tasks; limited by the GIL for CPU-bound work.
- **Multiprocessing** (multiprocessing module) — spins up separate processes, each with its own Python interpreter and memory space, bypassing the GIL; ideal for CPU-bound tasks.
- **Asyncio** — single-threaded, cooperative concurrency using an event loop and `async/await`; excellent for high-volume I/O-bound work without the overhead of threads or processes.

14. Decorators

A decorator is a function that wraps another function to extend its behavior without modifying its source code.

```
import functools
import time

def timer(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        print(f'{func.__name__} took {time.time() - start:.4f}s')
        return result
    return wrapper

@timer
def slow_function():
    time.sleep(1)
```

@timer above def slow_function is shorthand for slow_function = timer(slow_function). functools.wraps preserves the original function's name/docstring.

15. Context managers

```
class FileManager:
    def __init__(self, filename, mode):
        self.filename, self.mode = filename, mode

    def __enter__(self):
        self.file = open(self.filename, self.mode)
        return self.file

    def __exit__(self, exc_type, exc_val, exc_tb):
        self.file.close()

with FileManager("test.txt", "w") as f:
    f.write("hello")
```

__enter__ runs on entering the with block, __exit__ runs on exit — even if an exception occurs — making it ideal for resource cleanup (files, locks, DB connections). Can also be written with @contextlib.contextmanager and a generator function.

16. Variable scope — the LEGB rule

Python resolves names in this order: Local → Enclosing → Global → Built-in.

```
x = "global"

def outer():
    x = "enclosing"
    def inner():
        x = "local"
        print(x)  # "local"
    inner()

outer()
```

Use global or nonlocal keywords to modify variables outside the local scope.

17. Closures and the late-binding gotcha

A closure is a function that remembers values from its enclosing scope even after that scope has finished executing.

```
def make_multiplier(n):
    def multiplier(x):
        return x * n
    return multiplier

times3 = make_multiplier(3)
print(times3(10)) # 30
```

Classic gotcha — late binding in loops:

```
funcs = [lambda: i for i in range(3)]
print([f() for f in funcs]) # [2, 2, 2], NOT [0, 1, 2]
```

This happens because `i` is looked up at call time, not definition time, and all lambdas share the same `i`. Fix with a default argument:

```
funcs = [lambda i=i: i for i in range(3)]
print([f() for f in funcs]) # [0, 1, 2]
```

18. Metaclasses

A metaclass is the “class of a class” — it controls how classes themselves are created. `type` is the default metaclass in Python.

```
class Meta(type):
    def __new__(mcs, name, bases, namespace):
        namespace['greeting'] = 'Hello'
        return super().__new__(mcs, name, bases, namespace)

class MyClass(metaclass=Meta):
    pass

print(MyClass().greeting) # "Hello"
```

Rarely needed in day-to-day work, but used by frameworks (e.g. Django ORM, ABCs) for advanced class customization.

19. Monkey patching

Dynamically modifying or extending a class/module at runtime.

```
class Dog:
    def bark(self):
        return "Woof"

def new_bark(self):
    return "Woof Woof!"

Dog.bark = new_bark
print(Dog().bark()) # "Woof Woof!"
```

Useful for testing (mocking) but risky in production code since it can make behavior harder to trace.

20. @property vs a regular attribute

```
class Circle:
    def __init__(self, radius):
        self._radius = radius

    @property
    def radius(self):
        return self._radius
```

```

@radius.setter
def radius(self, value):
    if value < 0:
        raise ValueError("Radius can't be negative")
    self._radius = value

@property
def area(self):
    return 3.14159 * self._radius ** 2

```

@property lets you define methods that are accessed like attributes (circle.radius, not circle.radius()), enabling validation, computed values, and encapsulation while keeping a clean attribute-style API.

Section 3: OOP-Specific

21. Method Resolution Order (MRO) and multiple inheritance

Python uses the **C3 linearization algorithm** to determine the order in which base classes are searched when resolving a method or attribute.

```

class A:
    def greet(self): return "A"

class B(A):
    def greet(self): return "B"

class C(A):
    def greet(self): return "C"

class D(B, C):
    pass

print(D().greet())           # "B"
print(D.__mro__)             # (D, B, C, A, object)

```

Check the MRO with `ClassName.__mro__` or `ClassName.mro()`.

22. Abstract base classes

```

from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

class Square(Shape):
    def __init__(self, side):
        self.side = side

    def area(self):
        return self.side ** 2

```

Shape cannot be instantiated directly; any subclass **must** implement `area()`, or it can't be instantiated either. Useful for enforcing an interface/contract.

23. Composition vs inheritance

- **Inheritance** — “is-a” relationship (Car is a Vehicle).
- **Composition** — “has-a” relationship (Car has an Engine).

```
class Engine:
    def start(self):
        return "Engine started"

class Car:
    def __init__(self):
        self.engine = Engine()    # composition

    def start(self):
        return self.engine.start()
```

The general guidance is “favor composition over inheritance” — it produces more flexible, loosely-coupled code and avoids deep, fragile inheritance hierarchies.

24. Dunder (magic) methods

```
class Vector:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __add__(self, other):
        return Vector(self.x + other.x, self.y + other.y)

    def __eq__(self, other):
        return self.x == other.x and self.y == other.y

    def __len__(self):
        return int((self.x ** 2 + self.y ** 2) ** 0.5)

    def __repr__(self):
        return f"Vector({self.x}, {self.y})"
```

Common ones: `__init__`, `__repr__`, `__str__`, `__eq__`, `__lt__`, `__hash__`, `__len__`, `__getitem__`, `__setitem__`, `__call__`, `__enter__`/`__exit__`.
Implementing these lets custom objects work naturally with built-in operators and functions (+, ==, len(), in, etc.).

Section 4: Coding / DSA Questions

25. Reverse a string/list without built-ins

```
def reverse_string(s):
    result = ""
    for ch in s:
        result = ch + result
    return result

# Or, two-pointer in-place for a list:
def reverse_list(lst):
    left, right = 0, len(lst) - 1
    while left < right:
        lst[left], lst[right] = lst[right], lst[left]
        left += 1
        right -= 1
    return lst
```

26. Find duplicates in a list

```
def find_duplicates(nums):
    seen, duplicates = set(), set()
    for n in nums:
        if n in seen:
            duplicates.add(n)
        seen.add(n)
    return list(duplicates)
```

This runs in $O(n)$ time using a set for $O(1)$ average lookups.

27. FizzBuzz

```
for i in range(1, 101):
    if i % 15 == 0:
        print("FizzBuzz")
    elif i % 3 == 0:
        print("Fizz")
    elif i % 5 == 0:
        print("Buzz")
    else:
        print(i)
```

28. Flatten a nested list

```
def flatten(lst):
    result = []
    for item in lst:
        if isinstance(item, list):
            result.extend(flatten(item))
        else:
            result.append(item)
    return result

flatten([1, [2, 3, [4, 5]], 6]) # [1, 2, 3, 4, 5, 6]
```

29. Implement an LRU cache

```
from collections import OrderedDict

class LRUCache:
    def __init__(self, capacity):
        self.cache = OrderedDict()
        self.capacity = capacity

    def get(self, key):
        if key not in self.cache:
            return -1
        self.cache.move_to_end(key)
        return self.cache[key]

    def put(self, key, value):
        if key in self.cache:
            self.cache.move_to_end(key)
        self.cache[key] = value
        if len(self.cache) > self.capacity:
            self.cache.popitem(last=False) # evict least recently
used
```

Python also has `functools.lru_cache` as a ready-made decorator for memoizing function calls.

30. Two-sum

```
def two_sum(nums, target):
```

```

seen = {}
for i, n in enumerate(nums):
    complement = target - n
    if complement in seen:
        return [seen[complement], i]
    seen[n] = i
return []

```

O(n) time using a hash map instead of the O(n²) brute-force nested loop.

31. Anagram check

```

def is_anagram(s1, s2):
    return sorted(s1) == sorted(s2)
    # or: Counter(s1) == Counter(s2) (from collections import
Counter)

```

32. Palindrome check

```

def is_palindrome(s):
    s = s.lower().replace(" ", "")
    return s == s[::-1]

```

33. Sorting algorithm implementation (example: quicksort)

```

def quicksort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    mid = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quicksort(left) + mid + quicksort(right)

```

34. Simple decorator: memoization

```

def memoize(func):
    cache = {}
    def wrapper(*args):
        if args not in cache:
            cache[args] = func(*args)
        return cache[args]
    return wrapper

@memoize
def fib(n):
    if n <= 1:
        return n
    return fib(n - 1) + fib(n - 2)

```

Section 5: Practical / Applied

35. Working with pandas / numpy (data roles)

```

import pandas as pd

df = pd.read_csv("data.csv")
df.head()
df.groupby("category")["value"].mean()

```

```
df[df["value"] > 100]
df.fillna(0, inplace=True)
```

Interviewers commonly probe: loc vs iloc, vectorized operations vs apply(), handling missing data, merging/joining DataFrames, and why numpy arrays are faster than plain Python lists (contiguous memory, fixed type, vectorized C operations under the hood).

36. Writing unit tests

```
import unittest

class TestMath(unittest.TestCase):
    def test_addition(self):
        self.assertEqual(1 + 1, 2)

if __name__ == "__main__":
    unittest.main()
```

With pytest (more common in modern codebases):

```
def test_addition():
    assert 1 + 1 == 2
```

Interviewers may also ask about mocking (unittest.mock), fixtures, and test coverage.

37. Virtual environments and dependency management

```
python -m venv venv
source venv/bin/activate      # Linux/Mac
pip install -r requirements.txt
pip freeze > requirements.txt
```

Modern alternative: poetry or pipenv manage dependencies and virtual environments together, with lock files for reproducible installs.

38. REST API basics (Flask/FastAPI)

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/items/{item_id}")
def read_item(item_id: int):
    return {"item_id": item_id}
```

Interviewers may ask about HTTP methods (GET/POST/PUT/DELETE), status codes, request validation, and authentication basics.

39. SQL + Python integration (ORMs)

```
from sqlalchemy import create_engine, Column, Integer, String
from sqlalchemy.orm import declarative_base, sessionmaker

Base = declarative_base()

class User(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True)
    name = Column(String)

engine = create_engine("sqlite:///app.db")
```

```
Session = sessionmaker(bind=engine)
session = Session()
session.query(User).filter(User.name == "Alice").first()
```

40. Debugging / profiling

```
import cProfile
cProfile.run('my_function()')

# or step-through debugging:
import pdb
pdb.set_trace() # or the built-in breakpoint()
```

Section 6: Behavioral / Design (Senior Roles)

41. Design a rate limiter / cache / URL shortener

These are open-ended **system design** questions rather than pure coding ones. For a rate limiter, interviewers typically want you to discuss algorithms like **token bucket** or **sliding window log**, where to store state (in-memory dict for a single instance vs Redis for distributed systems), and trade-offs around accuracy vs performance. For a URL shortener, expect a discussion of hashing/base62 encoding, collision handling, and database schema. Walk through requirements, then a high-level design, then dig into one component in more depth.

42. How would you structure a large Python project

Typical structure:

```
myproject/
├── src/
│   └── myproject/
│       ├── __init__.py
│       ├── models/
│       ├── services/
│       ├── api/
│       └── utils/
├── tests/
├── requirements.txt
├── pyproject.toml
└── README.md
```

Key points to mention: separation of concerns (models/services/API layers), dependency injection over global state, consistent use of type hints, `pyproject.toml` for packaging, and a clear tests directory mirroring the source layout.

43. Code review — spot the bug

A favorite “gotcha” snippet interviewers use:

```
def process(items, results=[]): # BUG: mutable default argument
    for item in items:
        results.append(item * 2)
    return results
```

Calling `process([1, 2])` twice will **not** reset results between calls — it accumulates from the previous call, because the default list is created once, at function definition time. (See Question 2 for the fix.)

Quick-Reference Tips for the Interview

- Be ready to **write code on a whiteboard or shared doc** without an IDE — practice without autocomplete.
- For coding questions, always state the **time and space complexity** of your solution.
- If you don't know something, reason through it out loud — interviewers usually care more about your thought process than a perfect memorized answer.
- Practice explaining trade-offs, not just definitions (e.g. “why choose multiprocessing over threading here?”).
- For senior roles, expect at least one **system design** or **architecture** question in addition to coding.